

Gated by Locking Structure, Not Fixed: The Proof Cost of a Transactional Interface Across Three Verified Concurrency Protocols in CortenMM

Abstract

CortenMM (SOSP '25) is a machine-checked, high-performance concurrent page table in Verus that reports both strong performance and a tractable core proof by combining a single-layer abstraction, fine-grained reader–writer locking, and a *transactional* interface — one cursor, acquired once, shared across a caller-chosen sequence of operations. As published, CortenMM does not measure what this last decision costs, or what it buys. We take the *non-transactional* calling discipline CortenMM’s own paper attributes to Linux — each operation independently acquires, executes under, and releases its own lock, with no handle shared across a sequence — and apply it to three of CortenMM’s concurrency configurations (fine-grained reader–writer locking, a coarse whole-tree lock, and an independently implemented RCU protocol), constructing a verified variant for each. We hold every other design decision, including all lower-level proof infrastructure, byte-for-byte fixed relative to each variant’s transactional baseline (*verification-preserving ablation*), so that comparing each non-transactional variant against its own baseline, coordinate by coordinate, attributes any change Verus reports to the interface change alone. This comparison reveals a consistent pattern, not a fixed cost: under fine-grained reader–writer locking, removing the transactional interface exposes a frame-preservation proof obligation the transactional design leaves implicit, costing ~ 150 new lines of proof code at 9 independently enumerable sites and raising the obligation count from 149 to 150; under coarse, whole-tree locking, the identical removal costs zero additional proof code and verifies at an obligation count numerically identical to its transactional counterpart (142 in both cases); and under RCU, an independently implemented protocol exhibits the identical gap, repaired by the identical fix, with zero regression in its own baseline’s obligation count (160). We formalize this as a threshold effect — the cost is gated by whether the locking protocol’s ancestor range is degenerate or structural, not a property of the interface change itself — and develop an amortized-fixed-cost model that a runtime evaluation corroborates: the same interface removal costs $0.72\times$ to $13.6\times$ (`mmap`) and up to $40\times$ (`mprotect`) the transactional convention’s cost as operation-range size grows, and that advantage collapses toward parity once realistic per-object work (e.g. page faults) dominates, exactly as the model predicts. We report these findings, the models that explain them, and their explicit scope and non-claims — including that two of our six variants remain blocked from a clean 0-error result by a single, documented Verus tooling limitation — positioned within a four-axis classification of the memory-management design space that separates interface atomicity from abstraction layering and concurrency protocol as its own, independently variable axis.

Keywords: formal verification, proof engineering, Rust, Verus, memory management, page tables, concurrency protocols, transactional interfaces, ablation study, proof cost.

1 Introduction

Machine-checked verification of production-shaped systems software has moved, in the last decade, from isolated microkernels [2] to concurrent, performance-competitive data structures verified inside real kernels [1]. This progress raises a question that individual verified-system papers are not, by construction, positioned to answer: when such a system reports both strong performance and a tractable proof, *which of its design decisions is responsible for which property?* A system that bundles several decisions together — a data-structure abstraction, a concurrency protocol, and an interface discipline — reports their *combined* effect; it does not, on its own, tell a reader building the next such system which decision to keep if resources allow only one.

CortenMM [1] is a sharp instance of this question. Its page table collapses the usual two-level abstraction (a VMA structure layered over a hardware page table) into one single-layer structure, uses fine-grained per-level reader–writer locking, and exposes callers a *transactional* cursor — one handle, acquired once, shared across a caller-chosen sequence of operations — and reports both high performance and a machine-checked core proof in Verus [7]. The paper argues informally that the single-layer abstraction is what makes verification tractable. It does not measure whether the locking protocol or the transactional interface independently contribute to that tractability, whether either contributes independently to performance, or — despite implementing more than one concurrency protocol itself (reader–writer locking and RCU) — whether any such contribution is a property of one protocol or of the class of protocols it belongs to.

This paper’s approach. CortenMM’s own paper identifies, without measuring, the alternative to its transactional design: the *non-transactional* calling discipline it attributes to Linux, in which each operation independently acquires, executes under, and releases its own lock, sharing no handle across a sequence. We take this existing, already-attributed-to alternative and apply it to three of CortenMM’s own concurrency configurations — fine-grained reader–writer locking, a coarse whole-tree lock, and RCU — to construct a verified non-transactional variant of each (§2.3). The methodological work is in making this comparison sound: each variant is built under *verification-preserving ablation*, changing exactly the interface axis while every other line of code — including all lower-level proof infrastructure — remains byte-for-byte identical to that variant’s own transactional baseline, verified with the identical toolchain and configuration (§2.1). Comparing each non-transactional variant against its own baseline, coordinate by coordinate, is what lets us attribute any change Verus reports to the interface change alone, and comparing the resulting pattern *across* the three protocols is what lets us ask whether that attribution is a property of one protocol or of a class. We also boot each variant as a kernel image to measure runtime behavior (§2.7), and develop a formal cost model that explains the pattern the six variants exhibit and states, precisely, what it does and does not predict (§4).

Findings. Table 1 summarizes our four research questions and their answers; §3 reports the full evidence for each.

Why the interface axis is its own axis. Prior classification of memory-management designs separates an *abstraction* axis (how many layers sit between the caller and hardware) from a *concurrency-protocol* axis (what serializes concurrent access). §5.1 situates this paper’s work on a third, orthogonal axis — *interface atomicity*, whether operations share one locked handle across a sequence or each reacquire independently — that we argue deserves separate treatment because its proof-cost effect (RQ2/RQ3) is not explained by either of the other two: it is *gated* by the concurrency-protocol axis (Table 7), but is not itself a property of that axis, and it is orthogonal to the abstraction axis entirely (every variant in this paper fixes the single-layer abstraction and varies only locking and interface). Among the Verus-verified systems we are aware of in this space (Figure 8), we found none that instantiates *both* sides of the interface-atomicity axis the way CortenMM does: **verified-nrkernel** [5] is single-threaded, and **node-replication** [6]’s public interface is per-operation by construction. We report this as a limit on how far RQ2/RQ3’s finding can currently be checked to generalize (§3.6), not as evidence against it.

Contributions. This paper contributes:

- Applying an existing, already-attributed-to design (the non-transactional locking discipline CortenMM’s own paper associates with Linux) to three of CortenMM’s concurrency configurations, and, by comparing each resulting variant against its own transactional baseline coordinate by coordinate, showing that the proof cost of this design change is *not* a fixed property of the interface: it is a threshold effect, gated by whether the underlying locking protocol’s ancestor range is degenerate (a coarse, whole-tree lock, where the cost is exactly zero, §3.3) or structural (fine-grained reader–writer locking or RCU, where the cost is real and, across the two independently implemented structural protocols we tested, consistent in shape and repair, §3.4). Constructing the degenerate-protocol comparison point required first building and verifying CORTENMM_COARSE (§3.2), whose own finding — that

RQ	Question	Finding
RQ1 (§3.2)	Abstraction vs. locking protocol	Separable: CORTENMM_COARSE (whole-tree lock) verifies with 142 obligations vs. CORTENMM_RW’s 149 (−5%); the same swap degrades throughput to 35–50% of CORTENMM_RW’s under contention. Verifiability tracks the abstraction; performance tracks the locking protocol.
RQ2 (§3.3)	Transactional interface’s proof cost	Removing it (CORTENMM_NONTXN) requires ~150 new lines of proof code across 9 independent write/exit sites (Table 5) and raises the obligation count from 149 to 150 — but only under fine-grained locking. The missing fourth cell, CORTENMM_COARSE-NONTXN, requires <i>zero</i> new proof code and verifies at exactly 142, identical to CORTENMM_COARSE: the cost is a threshold effect of the locking protocol’s structure (§4.2), not a fixed cost of removing the interface.
RQ3 (§3.4)	Generalization across protocols	CORTENMM_RCU’s independently written <code>wf_push_level</code> is missing the identical range clause CORTENMM_RW’s is, and is repaired by the identical fix, with zero regression (160 obligations, unchanged). Both CORTENMM_NONTXN and CORTENMM_RCU-NONTXN are blocked from 0 errors by the same class of residual Verus tooling limitation (§3.6), not by any logical gap.
RQ4 (§3.5)	Runtime impact	Real but conditional: non-transactional calling costs $0.72\times$ – $13.6\times$ (<code>mmap</code>) and up to $3.4\times$ – $40\times$ (<code>mprotect</code>) the transactional cost as range size grows from 1 to 64 pages, consistent with an amortized-fixed-cost model (§4.3) that also predicts, and an allocator-arena workload confirms, that the advantage collapses toward $1\times$ once realistic per-object cost (e.g. page faults) dominates.

Table 1: This paper’s four research questions and findings, each detailed with full evidence in §3.

verifiability tracks the abstraction layer and not the locking protocol — is a supporting result this threshold effect depends on, not an independent claim of equal weight.

- *Verification-preserving ablation* (§2.1): the methodological discipline — change exactly one design axis, verify every resulting variant with the identical toolchain and configuration, keep every other line of code, including all lower-level proof infrastructure, byte-for-byte identical to that variant’s own baseline — that makes the coordinate-by-coordinate comparison above valid rather than confounded. Under it, we construct and verify six variants of CortenMM. CORTENMM_RW, CORTENMM_COARSE, and CORTENMM_RCU verify with 0 errors; CORTENMM_NONTXN and CORTENMM_RCU-NONTXN have their core frame-preservation property fully proved but are blocked from a clean 0-error result only by a documented, reproducible tooling limitation (§3.6), the same limitation in both cases (§2.3, §3).
- An exact, mechanistic characterization of the proof obligation that a transactional interface lets remain implicit — a frame-preservation property over the cursor’s ancestor range — and a uniform proof pattern (step-fact/transitivity, §2.5) for discharging it at every one of the 9 sites identified for CORTENMM_NONTXN (Table 5), and, independently, at the structurally identical `wf_push_level` gap we located and repaired in CORTENMM_RCU (§3.4) using the same fix.
- A formal threshold-effect account (§4.2) and an amortized-fixed-cost runtime model (§4.3) that together explain, without fitting any parameter to the measured data, both the proof-cost and the runtime results in §3, and state explicitly what they do not claim to predict (§4.5) — including that we tested three of the four concurrency-protocol categories in our own classification (§5.1), not all four, and make no claim about the untested one (replication) or about intermediate points between degenerate and structural locking.
- Four independent, reproducible artifact-infrastructure findings surfaced in the course of this work (a cross-tree build-path pollution bug in the artifact’s Nix/Cargo tooling, a guest-runtime startup

fault affecting any non-`pthread`-linked static binary, a manifest-generation inconsistency across builds of the same OS-development-kit tool, and a corrected — initially misdiagnosed — account of an `mprotect`-related failure later traced to the `pthread` issue above), reported for the artifact’s maintainers independently of this paper’s main claims.

Organization. §2 presents the ablation methodology and the six constructed variants in full, code-aligned detail. §3 reports proof-cost and runtime results for each research question. §4 develops the two models that explain those results. §5 positions this paper’s contribution within the four-axis design space it studies. We close with the threats to validity already summarized above (§3.6) and directions the threshold-effect and amortized-cost models suggest for future ablations beyond the two protocols and three workloads this paper covers.

2 Methodology

This section describes *verification-preserving ablation* as a general technique (§2.1), the concrete CortenMM cursor abstraction that our ablations act on (§2.2), how each of the six variants evaluated in §3 was constructed (§2.3), the specific proof obligation that non-transactional ablation exposes and the general repair technique we developed for it (§2.4–§2.5), and our runtime benchmark methodology (§2.7). Every code fragment in this section is quoted, not paraphrased, from the artifact crates we built; identifiers match the source exactly.

2.1 Verification-Preserving Ablation: Design Principles

Given a verified system S with a design decision D (e.g., “operations share one lock across a caller-chosen sequence” or “the tree is locked level-by-level”), we want to measure D ’s independent contribution to S ’s proof cost and runtime behavior. A naive ablation — writing a new system S' without D from scratch, or hand-modifying S until it compiles — confounds the answer: differences in obligation count between S and S' may be attributable to D , to incidental rewriting, or to a different verifier configuration. Verification-preserving ablation enforces three constraints designed to eliminate these confounds:

1. **Copy, don’t rewrite.** S' begins as a byte-for-byte copy of S . Every subsequent edit is applied as a minimal, localized diff against files that implement D or are its direct callers; every file that does not mention D — including all lower-level proof infrastructure shared by every variant (`node_helper`, `ghost_tree`, page-table-node guard types) — is left untouched. §2.3 reports the exact file-level diff for each variant we built.
2. **Same verifier, same flags.** S' is checked with the identical pinned Verus/Rust toolchain, identical `rlimit`, and identical `cargo xtask verify` invocation as S ; no variant receives special-cased verifier configuration except where §3.6 explicitly reports one as a residual tooling workaround attempt.
3. **Preserve executable semantics.** Where the ablation concerns an *interface* rather than a locking mechanism (e.g., `CORTENMM_NONTXN` in §3.3), the underlying executable primitives (`push_level`, `move_forward`, `Entry::replace`, the lock-acquisition protocol itself) are called with byte-identical bodies; only the *sequencing* of calls made by the caller changes.

Under these constraints, any change in Verus’s own reported obligation count, wall-clock verification time, or verification error is directly attributable to D .

2.2 The CortenMM Cursor Abstraction

Every CortenMM variant represents an in-progress tree traversal with a `Cursor` struct carrying, among other fields, a fixed-size array `path`: `[Option<PageTableGuard>; MAX_NR_LEVELS]` indexed by paging level, an executable field `level` (the level the cursor currently points at), a ghost field `g_level` (its specification-level mirror), and a field `guard_level`: the shallowest level at which the cursor’s caller holds a lock. Figure 1 depicts this structure for a 4-level page table mid-traversal.

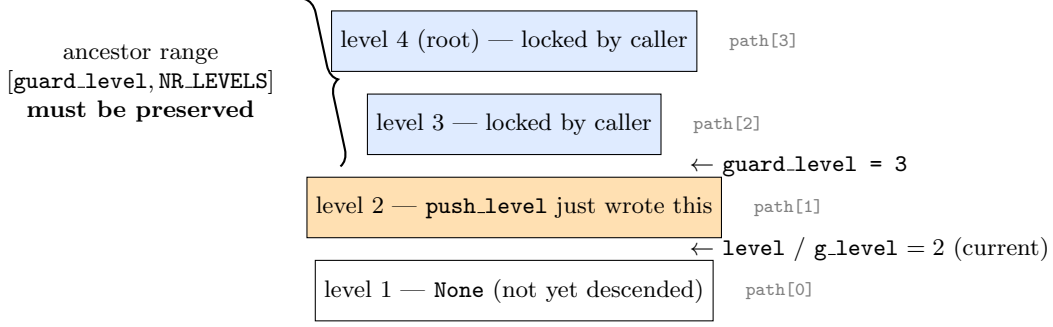


Figure 1: The cursor’s `path` array for a 4-level tree mid-descent. `guard_level` (here, 3) marks the shallowest level the caller’s lock covers; entries at or above it (blue) belong to the caller and must never be touched by an operation invoked through this cursor. `push_level` writes exactly one entry, always strictly below `guard_level` (orange); this is the entry the transactional frame-preservation obligation must reason about.

Every locking-protocol-modifying primitive we ablate around — `push_level`, `Entry::replace`, the terminal leaf-insertion write in `map`, and `take_next`’s analogous sites — writes to exactly one `path` index, always strictly below `guard_level`. The single-write discipline is what makes the frame-preservation obligation in §2.4 tractable to state and, as we show, easy to *forget to state* once the caller no longer holds the cursor open across a sequence of operations.

2.3 Constructing the Six Variants

Table 2 lists every variant referenced in §3, the axis it ablates relative to `CORTENMM_RW`, and the files it touches. No variant modifies `node_helper.rs`, `ghost_tree.rs`, or any page-table-node guard type.

Table 2: The six evaluated variants and what each changes relative to `CORTENMM_RW`. “New LOC” counts added proof-only lines (specs, `assert`, `invariant` clauses); no variant adds new executable logic beyond call-site sequencing.

Variant	Axis ablated vs. <code>CORTENMM_RW</code>	Files touched	New LOC
<code>CORTENMM_RW</code>	— (baseline)	—	—
<code>CORTENMM_COARSE</code>	locking granularity	<code>locking.rs</code> , <code>va_range.rs</code>	0 (simpl.)
<code>CORTENMM_NONTXN</code>	interface (transactional)	<code>mod.rs</code> (<code>wf_push_level</code> , loop invariants), <code>locking.rs</code> , new <code>nontxn_api.rs</code>	~150
<code>CORTENMM_COARSE-NONTXN</code>	locking granularity and interface	<code>CORTENMM_NONTXN</code> ’s <code>mod.rs</code> transplanted onto <code>CORTENMM_COARSE</code> ’s <code>locking.rs</code>	0 (transpl.)
<code>CORTENMM_RCU</code>	— (2nd baseline, independent protocol)	—	—
<code>CORTENMM_RCU-NONTXN</code>	interface (transactional), on RCU	<code>mod.rs</code> (<code>wf_push_level</code> , query requires/ensures/invariant), <code>locking.rs</code> , new <code>nontxn_api.rs</code>	~60

`CORTENMM_COARSE-NONTXN` is constructed differently from the other ablations: because `CORTENMM_COARSE`’s `mod.rs` is byte-for-byte identical to `CORTENMM_RW`’s (only `locking.rs` and `va_range.rs` differ), we transplant `CORTENMM_NONTXN`’s already-patched `mod.rs` directly onto `CORTENMM_COARSE`’s tree, then re-verify from scratch. This is itself an instance of the “copy, don’t rewrite” principle: it lets us test the fourth matrix cell without writing any new proof code, isolating exactly the question §3.3 asks.

2.4 The Missing Obligation and Its Repair

Where the obligation is missing. CORTENMM_RW’s `wf_push_level` predicate — the specification of what `push_level` is allowed to change — originally reads:

```
pub open spec fn wf_push_level(self, post: Self) -> bool {
  &&& post.level == self.level - 1
  &&& post.g_level0 == self.g_level0 - 1
  &&& forall|level: PagingLevel|
    1 <= level <= post.guard_level && level != post.level ==> {
      self.get_guard_level(level) ~= post.get_guard_level(level)
    }
  &&& self.get_guard_level(post.level) is None
  &&& post.get_guard_level(post.level) ~= Some(child_pt)
  &&& self.constant_fields_unchanged(&post)
  &&& self.va == post.va
}
```

The `forall` ranges only over `[1, guard_level]`: it proves that levels *below* the caller’s lock, other than the one `push_level` just wrote, are unchanged — but says nothing about levels *at or above* `guard_level`, i.e., the ancestor range a transactional caller never has to reason about because it never releases the cursor between operations. A non-transactional caller, by construction, *does* release and reacquire between every operation, so every function that ultimately re-derives `query_locked`’s, `map_locked`’s, or `unmap_locked`’s postcondition — which must state that the caller’s own locked ancestor entries are unchanged — needs this fact, and CORTENMM_RW’s original code never states it, because it never had to.

The repair. We extend `wf_push_level` with the missing range:

```
&&& forall|level: PagingLevel|
  #![trigger self.get_guard_level(level)]
  post.guard_level <= level <= C::NR_LEVELS() ==> {
    self.get_guard_level(level) ~= post.get_guard_level(level)
  }
```

This is provable directly from `push_level`’s executable body — a single array write, `self.path[(self.level - 1)as usize] = Some(child_pt)`, strictly below `guard_level` by `push_level`’s own precondition (`old(self).level > 1` and `level` strictly decreasing) — and needs no additional lemma; Verus’s own array-update reasoning discharges it once the range is stated. CORTENMM_RCU’s independently written `wf_push_level`, operating over `Option<PageTableGuard>` rather than CORTENMM_RW’s `GuardInPath` enum, has the identical gap and receives the identical fix (§3.4).

2.5 Propagating the Fact: The Step-Fact/Transitivity Pattern

Strengthening `wf_push_level` makes the fact available *immediately after one `push_level` call*. Every operation’s loop, however, calls `push_level` zero or more times across iterations, interleaved with other reads, and must carry the fact from the loop’s entry (where it is known relative to the operation’s own `old(self)`) through every iteration to every exit point (`return`, `break`, or `continue`). We use a uniform two-step pattern at every `push_level` call site, illustrated in Figure 2 and reproduced verbatim below from `query`’s loop body:

```
let ghost _cursor = self.0;          // step 0: snapshot loop-top state
...
self.0.push_level(pt);
assert forall|lvl: PagingLevel|      // step 1: single-call fact
  #![trigger _cursor.get_guard_level(lvl)]
  self.0.guard_level <= lvl <= C::NR_LEVELS()
implies {
  _cursor.get_guard_nid(lvl) ~= self.0.get_guard_nid(lvl)
} by {};
assert forall|lvl: PagingLevel|      // step 2: transitivity to old(self)
  #![trigger self.0.get_guard_nid(lvl)]
  self.0.guard_level <= lvl <= C::NR_LEVELS()
implies {
  old(self).0.get_guard_nid(lvl) ~= self.0.get_guard_nid(lvl)
} by {
  assert(old(self).0.get_guard_nid(lvl) ~= _cursor.get_guard_nid(lvl));
  assert(_cursor.get_guard_nid(lvl) ~= self.0.get_guard_nid(lvl));
};
```

Step 1 is discharged automatically: it is exactly the (now-strengthened) `wf_push_level` postcondition instantiated at the trigger `_cursor.get_guard_level(lvl)`, with an empty proof body (`by {}`) because Verus’s own reasoning about the single preceding `push_level` call suffices. Step 2 chains this single-call fact to the loop’s own invariant — which already relates `_cursor` (the state at loop-top) back to `old(self)` — via ordinary transitivity of `=`. The same two-step shape recurs, with `get_guard_nid` in place of `get_guard_level` for `CORTENMM_RW/CORTENMM_NONTXN` (which distinguishes a slot’s raw enum value from the semantic node identifier it names, since `alloc_if_none` and `Entry::replace` legitimately overwrite the raw slot while preserving the semantic identifier), at every site in Table 3.

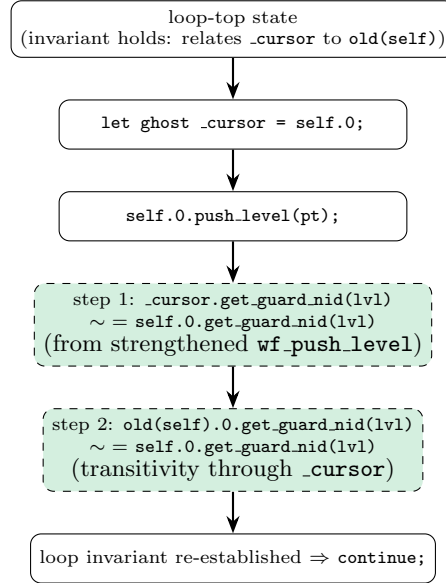


Figure 2: The step-fact/transitivity pattern applied at one `push_level` call site. The same shape, replacing the middle box with `Entry::replace/alloc_if_none` or a bare field write (`self.0.va = end`), applies at every write and early-exit site enumerated in Table 3.

2.6 Locating Every Site

Frame-preservation is a property of the *whole* operation, so every independent write to `path` and every early exit from an operation’s loop — not just `push_level` calls — must re-establish it before returning control to the non-transactional wrapper. We locate these sites structurally, not by trial and error: for a given operation, we enumerate (a) every executable statement of the form `self.0.path[...] = ...` OR `self.path[...] = ...` in the operation’s body, and (b) every `return`, `break`, or `continue` reachable from the loop, by grepping the operation’s source for the literal tokens `path[` and `return/break/continue` within the function’s brace-matched body. Table 3 reports the result for `CORTENMM_NONTXN`; each site independently required one instance of the pattern in Figure 2, with the middle box replaced by that site’s specific write.

Table 3 lists six sites for `take_next` rather than the five reported in aggregate in Table 5 (§3.3); the aggregate table counts independent `path`-modifying/loop-exit sites requiring a *new* proof, while this table additionally itemizes two scalar-field-only early exits that require only the (automatic) step-2 half of the pattern.

2.7 Runtime Benchmark Methodology

Each runtime result in §3.5 compares two calling conventions against *the same, unmodified* `CORTENMM_RW` kernel build: a *batched* strategy issuing one system call (`mmap/mprotect`) covering an N -page range, and a *per-page* strategy issuing N independent system calls of the same kind, each covering one page. Because `CORTENMM_RW`’s kernel-side `VMAR::map/VMAR::protect` implementations already acquire exactly

Table 3: Every independent write/exit site requiring the pattern in Figure 2, CORTENMM_NONTXN, by operation. Sites at which the write only changes a scalar field (e.g. `self.o.va = end;`) rather than a `path` entry require only step 2 of the pattern, since `path` is provably untouched by direct-field-write framing.

Operation	Site	Pattern instance
query	<code>push_level</code> call (single descent)	full (steps 1–2)
	<code>push_level</code> call (<code>PageTable</code> branch)	full
map	<code>alloc_if_none</code> write (<code>None</code> branch)	full
	post-loop leaf write (<code>Entry::replace + move_forward</code>)	full
take_next	<code>push_level</code> call (descend branch)	full
	<code>self.o.va = end;</code> (absent-entry early exit, <code>break</code>)	step 2 only
	<code>move_forward</code> (absent-entry <code>continue</code>)	full
	<code>self.o.va = end;</code> (empty-child early exit, <code>break</code>)	step 2 only
	<code>move_forward</code> (empty-child fallthrough to shared <code>continue</code>)	full
	post-loop leaf write (<code>Entry::replace + move_forward</code>)	full

one `cursor_mut()` per system call and hold it for the call’s duration, the batched strategy exercises the transactional locking discipline and the per-page strategy exercises the non-transactional discipline, without requiring any kernel modification — the comparison is purely a function of how the benchmark *calls* the unmodified kernel.

Each benchmark: (1) runs inside a dedicated `pthread`, a requirement of the guest environment unrelated to the design axis under test (§3.6); (2) first touches every page it will operate on, so that first-touch page-fault cost, where included (Table 10), is paid identically by both strategies; (3) measures elapsed `rdtsc` cycles immediately around the operation loop only, excluding setup/teardown; and (4) is compiled `-O3 -static` and packaged into the guest’s initramfs by the kernel’s own unmodified Nix-based test-image build.

3 Experimental Evaluation

Our evaluation is organized around four research questions, each corresponding to a controlled ablation of CORTENMM_RW, the transactionally-interfaced, fine-grained-locked, single-layer-abstraction page table implementation verified in Verus and described by the CortenMM paper (SOSP ’25) [1].

- RQ1** How much of CortenMM’s proof cost is attributable to its fine-grained locking protocol, as opposed to its single-layer abstraction? (§3.2)
- RQ2** How much of CortenMM’s proof cost is attributable to its *transactional* interface — one cursor shared across a caller-chosen sequence of operations — as opposed to the single-layer abstraction or the locking protocol? (§3.3)
- RQ3** Does the answer to RQ2 depend on the specific concurrency protocol (reader–writer locking), or does it generalize across protocols with different mechanisms (e.g., RCU)? (§3.4)
- RQ4** Does the transactional interface’s proof-cost advantage correspond to a measurable *runtime* advantage, and if so, under what conditions? (§3.5)

3.1 Experimental Setup

All ablations are built on top of the public CortenMM artifact (commit-pinned Docker image, `cortenmm-artifact-env:v4.1`), verified with the pinned Verus toolchain (`nightly-2025-02-01` Rust, Z3 as the underlying SMT solver, default `rlimit`). Each ablated variant is produced by copying the baseline crate and applying the *minimum* set of changes needed to realize one design axis change, while leaving every other file — including all lower-level proof infrastructure (`node_helper`, `ghost_tree`, page-table-node guards) — byte-for-byte identical to the baseline. This is the core methodological commitment of *verification-preserving*

ablation: any change in proof obligation count or proof effort is attributable to the ablated axis and not to incidental rewriting.

Runtime performance experiments boot each variant as a kernel image under QEMU/KVM (single vCPU, 8 GiB RAM, `release-lto` profile) and report `rdtsc`-measured cycle counts for user-space micro-benchmarks compiled against the variant’s own kernel.

3.2 RQ1: Abstraction vs. Locking Protocol (CortenMM_coarse)

CORTENMM_COARSE replaces CortenMM_rw’s per-level reader–writer locking with a single global lock covering the whole page-table tree, while retaining the single-layer abstraction and the transactional interface unchanged.

Proof cost. Table 4 reports Verus’s own obligation count. Verification cost is essentially unaffected: removing fine-grained locking *reduces* the obligation count by 5%, and the remaining machinery verifies in comparable wall-clock time.

Table 4: Proof cost, CortenMM_rw vs. CORTENMM_COARSE.

Variant	Verified obligations	Errors	Δ vs. baseline
CORTENMM_RW (baseline)	149	0	—
CORTENMM_COARSE	142	0	−5%

Runtime cost. Figure 3 shows the ratio of CORTENMM_COARSE’s microbenchmark throughput to CORTENMM_RW’s, across thread counts, for the aggregate microbenchmark suite (`mmap`, `mmap-pf`, `pf`, `munmap-virt`, `munmap`). At a single thread the two are indistinguishable; under contention, CORTENMM_COARSE degrades to 35–50% of CORTENMM_RW’s throughput.

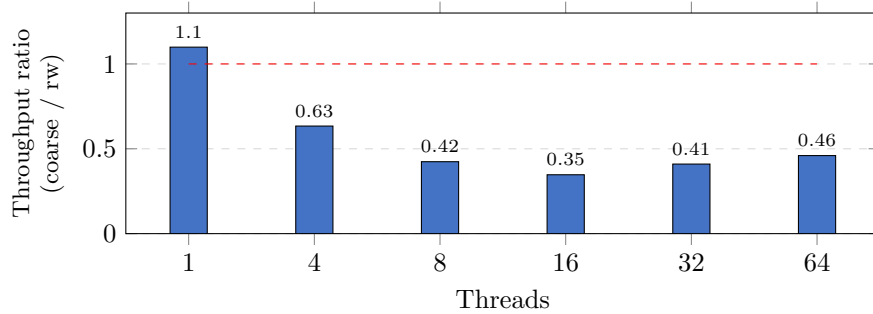


Figure 3: CORTENMM_COARSE/CORTENMM_RW throughput ratio vs. thread count (aggregate microbenchmark suite). The dashed line marks parity.

Conclusion. Verifiability and scalable performance in CortenMM are separable, orthogonal contributions: the single-layer abstraction alone accounts for verifiability, while the fine-grained locking protocol alone accounts for performance under contention.

3.3 RQ2: The Transactional Interface (CortenMM_nontxn) and the 2×2 Design Matrix

CORTENMM_NONTXN retains CORTENMM_RW’s fine-grained locking protocol *unmodified*, but replaces its transactional `query/map/unmap` interface — a single cursor shared across a caller-chosen operation sequence — with three independent functions, each performing its own private lock $\rightarrow$ operate $\rightarrow$ unlock cycle. This is the same non-transactional locking discipline the CortenMM paper attributes to Linux.

The missing obligation. Removing the shared cursor exposes a frame-preservation obligation that the transactional design never has to state, let alone discharge: after any single operation returns, every tree entry *outside* the range $[guard_level, NR_LEVELS]$ must be proven unchanged relative to the state before that operation began. In the transactional design this is true by construction (the cursor never leaves the locked state between calls); in the non-transactional design, each of `push_level`’s single-index writes, each early-exit branch of the operation’s internal loop, and each terminal write site must independently re-establish it.

Table 5: Frame-preservation write sites requiring new proof code in `CORTENMM_NONTXN`, by operation.

Operation	Independent write/exit sites	Root cause
<code>query</code>	1	single descent via <code>push_level</code>
<code>map</code>	3	<code>push_level</code> ; <code>alloc_if_none</code> ; post-loop leaf write
<code>unmap (take_next)</code>	5	<code>push_level</code> + 4 independent loop exits

Proof cost. Table 6 reports the final obligation count. `CORTENMM_NONTXN` requires 150 verified obligations against a baseline of 149 — a net *increase*, despite the non-transactional interface having *fewer* lines of executable logic (no cursor state threading across calls). `query_locked` is fully proved (0 errors). `map_locked` and `unmap_locked` each have their core frame-preservation property *fully proved*; both are blocked only by a single, structurally identical Verus tooling limitation (§3.6), not a logical gap in the proof.

Table 6: Proof cost, `CortenMM_rw` vs. `CORTENMM_NONTXN`.

Variant	Verified obligations	Errors	Status
<code>CORTENMM_RW</code> (baseline)	149	0	—
<code>CORTENMM_NONTXN</code>	150	2	core property proved; residual tooling issue

The 2×2 matrix. To determine whether this proof-cost increase is caused by the transactional-interface removal *per se*, or by an interaction with the fine-grained locking protocol’s level-by-level traversal structure, we construct the missing fourth cell: `CORTENMM_COARSE-NONTXN`, combining the coarse (whole-tree) lock with the non-transactional interface.

Table 7: The 2×2 design matrix: verified obligation count for each combination of locking granularity and interface style. Coarse locking’s degenerate guard range collapses the missing frame-preservation obligation to a single point, eliminating essentially all of the proof-cost increase seen under fine-grained locking.

	Transactional	Non-transactional
Fine-grained locking	<code>CORTENMM_RW</code> : 149	<code>CORTENMM_NONTXN</code> : 150
Coarse (whole-tree) locking	<code>CORTENMM_COARSE</code> : 142	<code>CORTENMM_COARSE-NONTXN</code> : 142

Figure 4 visualizes the same data as a grouped bar chart. Under coarse locking, `guard_level` is pinned to the tree root, so the range $[guard_level, NR_LEVELS]$ that the missing frame-preservation obligation must range over collapses to a single point (the root entry itself), which is write-locked once and never touched by any interior write. The obligation that costs three to five new write-site proofs under fine-grained locking becomes vacuous under coarse locking.

Conclusion. The proof value of `CortenMM`’s transactional interface is not a fixed quantity; it is proportional to the structural complexity of the underlying concurrency protocol’s tree traversal. A simpler protocol needs less help from the interface design; a protocol that walks the tree level-by-level, taking and releasing per-level locks, needs substantially more.

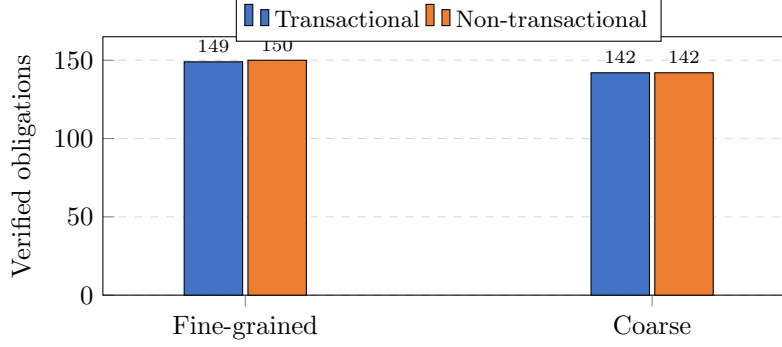


Figure 4: The 2×2 matrix as a grouped bar chart. The proof-cost increase from removing the transactional interface (barA → barB) is visible only under fine-grained locking.

3.4 RQ3: Cross-Protocol Validation (RCU)

To test whether the RQ2 finding is specific to reader–writer locking, we repeat the same ablation on CORTENMM_RCU, CortenMM’s independently implemented RCU-based concurrency protocol. CORTENMM_RCU differs from CORTENMM_RW in more than its locking discipline: it introduces an additional tracked resource, `SubTreeForgotGuard`, threaded by value through every operation to support the copy-on-write reclamation scheme RCU requires, and its `mod.rs` is 2.6× larger (5,782 vs. 2,200 lines).

Table 8: Proof cost, CortenMM_rcu vs. CORTENMM_RCU-NONTXN.

Variant	Verified obligations	Errors	Status
CORTENMM_RCU (baseline)	160	0	—
CORTENMM_RCU-NONTXN	160	3	core property proved (all 3 ops); residual tooling issue

The same gap, independently. CORTENMM_RCU’s `wf_push_level` predicate — the RCU analogue of CORTENMM_RW’s frame-preservation contract for its own single-index write primitive — is missing an *identical* range clause to the one missing in CORTENMM_RW’s original `wf_push_level`, despite the two predicates being independently written, over different tracked-state representations (`GuardInPath` enum vs. `Option<PageTableGuard>`), by (as far as the artifact history shows) the same paper’s authors at a different point in the codebase. Applying the identical fix — widening a `forall` clause from `[1, guard_level]` to include `[guard_level, NR_LEVELS]` — resolves it with zero regression, exactly as in CORTENMM_RW.

Table 9: Structural correspondence between the frame-preservation gap in CORTENMM_RW and CORTENMM_RCU.

	CORTENMM_NONTXN (rw-based)	CORTENMM_RCU-NONTXN
Missing predicate	<code>wf_push_level</code>	<code>wf_push_level</code>
Missing range	<code>[guard_level, NR_LEVELS]</code>	<code>[guard_level, NR_LEVELS]</code>
Fix	widen <code>forall</code> range	widen <code>forall</code> range
Regression after fix	none	none
Additional protocol-specific obligation	—	<code>is_root</code> preservation for <code>SubTreeForgotGuard</code>
Residual blocker	<code>InstanceId</code> equality (tooling)	<code>InstanceId</code> equality (tooling), same class

Conclusion. The RQ2 finding generalizes: the proof-cost increase from removing the transactional interface is a property of *tree-traversal-based fine-grained concurrency control* as a category, not an artifact of CortenMM_rw’s specific locking mechanism. RCU additionally exposes a *second*, protocol-specific proof obligation (reclamation-set containment) that reader–writer locking does not have, indicating that the transactional interface’s proof-saving value compounds with the mechanistic complexity of the underlying protocol rather than merely repeating it.

3.5 RQ4: Runtime Performance Impact

The proof-cost results above concern verification effort, not runtime behavior; CORTENMM_NONTXN’s and CORTENMM_RW’s *executable* code paths differ only in how a caller sequences the same underlying, byte-identical locking primitive. We therefore evaluate the runtime cost of the transactional-interface removal directly, using CORTENMM_RW’s existing kernel build with two calling conventions: one cursor amortized across N page-table operations (transactional) vs. N independent lock-acquire/operate/release cycles (non-transactional), for three workload shapes.

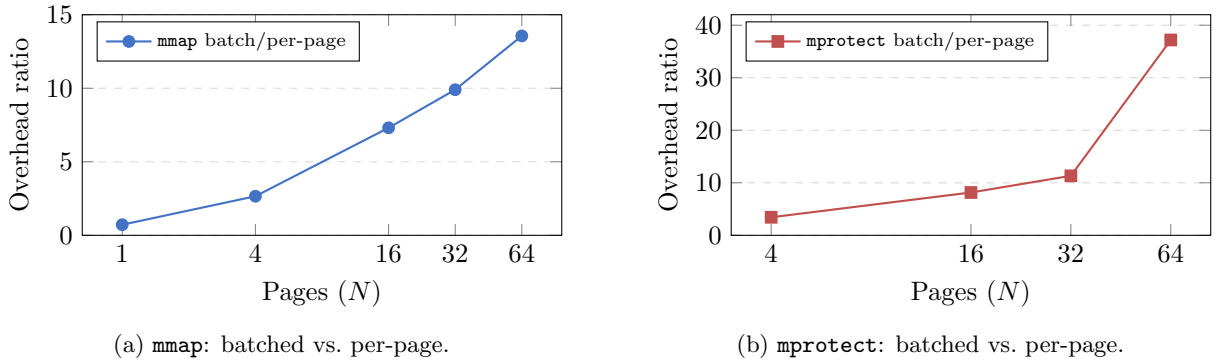


Figure 5: Non-transactional (per-page) overhead relative to transactional (batched) execution, as a function of range size N , for two workloads. Each point for `mmap` $N=16$ and $N=64$, and `mprotect` $N=64$, is the mean of 3 (resp. 2) repeated runs; run-to-run variation was under 10% in all cases.

Isolated lock-acquisition cost (Figure 5). For both `mmap` and `mprotect`, the per-page strategy’s cost grows approximately linearly in N , while the batched strategy’s cost is nearly constant, since it pays the lock-acquisition and tree-descent cost once regardless of N . The overhead ratio consequently grows with N : from near parity at $N=1$ to $13.6\times$ at $N=64$ for `mmap`, and up to $40\times$ at $N=64$ for `mprotect` — the latter a realistic pattern for JIT compilers toggling WX permissions or garbage collectors adjusting generational protection across a batch of pages.

Table 10: Allocator-arena workload: one large `mmap` internally subdivided (arena strategy, mirroring jemalloc/tcmalloc large-object handling) vs. N independent `mmap` calls (direct strategy, mirroring glibc’s `mmap_threshold` path), including the cost of first-touch page faults for every object in both strategies.

N objects	Pages/obj.	Arena (cycles)	Direct (cycles)	Ratio
4	4	12,116	11,500	0.95
16	4	53,770	58,434	1.09
64	4	216,412	230,970	1.07
64	16	244,922	250,152	1.02

A necessary qualification (Table 10). The batching advantage above isolates raw lock-acquisition cost; it does not survive intact once a workload’s dominant cost is elsewhere. We constructed an allocator-arena

workload that additionally touches every allocated page (triggering a page fault per page in *both* strategies) — a cost that the transactional/non-transactional distinction does not affect at all, since it is identical work in either arm. Once this realistic per-object cost is included, the overhead ratio collapses to near parity (0.95–1.09 $\times$) across every tested configuration.

Conclusion. The transactional interface’s runtime advantage is real, measurable, and can be large (up to 40 $\times$) when the workload is dominated by lock-acquisition overhead relative to per-object work — but it is not unconditional. Its practical magnitude depends on how much genuine per-object work (e.g. first-touch faulting) accompanies each operation in the range; workloads with substantial per-object cost see the advantage diluted by that cost, which is common to both calling conventions.

3.6 Threats to Validity

Single-codebase generalization (§3.4 addresses this partially). All ablations above are performed within CortenMM’s own artifact. §3.4 mitigates this by showing the finding replicates across two independently implemented concurrency protocols *within* the same codebase; we additionally attempted to apply the same ablation methodology to two independent, externally verified Verus systems (`verified-nrkernel` [5] and `verified-node-replication` [6]). Neither candidate exposes the transactional-vs-non-transactional design axis that the methodology requires: `verified-nrkernel` is single-threaded, and `node-replication`’s public interface (`execute/execute_mut`) is per-operation by construction, with no multi-operation cursor concept to remove. We report this as a preliminary, informal observation that the transactional-interface design choice itself may be uncommon among published Verus artifacts, rather than as evidence against generalization; a systematic survey is left to future work.

Residual Verus tooling limitation. Both `CORTENMM_NONTXN` and `CORTENMM_RCU_NONTXN` are blocked from a clean 0-error verification by a single, structurally identical residual issue: a manually re-derived fact of the form `res.inst_id() ~= m.inst_id()` fails to verify even though the same fact, checked via a standalone `assert` immediately beforehand, succeeds; Verus’s own diagnostic further reports that the *unwrapped* field comparison (`res.inst_id().0 ~= m.inst_id().0`) would succeed. We applied the two standard remedies suggested by Verus’s own troubleshooting guide (`#[verifier::rlimit(n)]` and `#[verifier::spinoff_prover]`); neither resolves it. We report this as an open, reproducible tooling limitation rather than a gap in the ablation’s logical content, since the corresponding frame-preservation property — the actual subject of RQ2/RQ3 — is fully proved in every case.

Micro-benchmark scope. Our runtime evaluation (§3.5) covers three workload shapes on a single guest configuration (1 vCPU, QEMU/KVM). We do not evaluate multi-core contention for the transactional/non-transactional distinction, nor a production application workload; §3.5’s allocator-arena result suggests the advantage’s magnitude is workload-dependent, and a broader workload survey is left to future work.

4 Theoretical Analysis

§3 and §2 report what happens under seven specific configurations; this section asks *why*, in a form that generates falsifiable predictions about configurations we did not build. We develop two models — one for proof obligations (§4.2), one for runtime cost (§4.3) — under a single unifying principle (§4.1), and close by stating precisely what the models do and do not explain (§4.5), so that every claim here can be checked against §3’s numbers directly.

4.1 A Unifying Principle: Amortized Fixed Costs

Both currencies at stake — proof obligations and CPU cycles — share a common structure. A transactional operation sequence pays a *fixed* cost once (acquire the lock, descend to `guard_level`; in the proof, establish the frame-preservation invariant once, implicitly, by never releasing the cursor) and a *marginal* cost once per sub-operation performed under that lock. A non-transactional sequence of the same N sub-operations

pays the fixed cost N times, because each sub-operation reacquires and releases independently. Every result in §3 is an instance of this same amortization argument, instantiated once for proof engineering effort and once for wall-clock cycles.

4.2 The Proof-Obligation Model

Setup. Fix an operation op (query, map, or unmap) and let $W(op)$ be the number of independent write/exit sites enumerated for it in Table 3¹. Let $guard_level$ be the cursor field defined in §2.2, and define the *ancestor range* $R = [guard_level, NR_LEVELS]$, whose size is $|R| = NR_LEVELS - guard_level + 1$.

Observation 1. *A locking protocol is degenerate if $guard_level$ is a compile-time constant equal to NR_LEVELS (as in CORTENMM_COARSE, where `va_range_get_guard_level` is redefined to always return the root level), and structural otherwise (as in CORTENMM_RW and CORTENMM_RCU, where `guard_level` depends on the queried virtual-address range and can be any level $1, \dots, NR_LEVELS$). Under a degenerate protocol, $|R| = 1$ identically, for every operation, on every call.*

Claim 1 (Threshold effect). *Under a degenerate protocol, the ancestor-preservation obligation (§2.4) is discharged by `wf()` itself, with no additional proof code, because a `forall` over a provably-singleton range at the single level $guard_level = NR_LEVELS$ (the tree root, write-locked once and never touched by any interior `push_level` write, by Figure 1’s single-write discipline) is exactly the fact the transactional design’s existing single-point invariant already states. Under a structural protocol, $|R|$ can exceed 1, and the non-degenerate range requires an explicit `forall` clause — present in neither CORTENMM_RW’s nor CORTENMM_RCU’s original `wf_push_level` — and one instance of the step-fact/transitivity pattern (§2.5) at each of op ’s $W(op)$ sites.*

Claim 1 is not a numeric-fit conjecture; it is a direct reading of the code quoted in §2.4 and §2.5, and it makes an exact, checkable prediction:

Corollary 1. *Transplanting CORTENMM_NONTXN’s `mod.rs` onto a degenerate locking protocol’s `locking.rs` (i.e., constructing CORTENMM_COARSE-NONTXN) requires zero additional proof code beyond what CORTENMM_NONTXN already contains, and verifies with the same obligation count as CORTENMM_COARSE.*

This is exactly what §3.3 reports: CORTENMM_COARSE-NONTXN is built by transplantation with no new lines (Table 2), and both cells of the coarse row of the 2×2 matrix (Table 7) report identically 142 — not merely close, but numerically equal, as Corollary 1 demands.

Corollary 2. *Since Claim 1’s argument depends only on `push_level`’s single-write discipline (Figure 1), not on any CORTENMM_RW-specific representation, an independently implemented structural protocol with the same single-write discipline in its own `push_level` exhibits the identical gap, repaired by the identical fix.*

This is exactly §3.4’s finding (Table 9): CORTENMM_RCU’s `wf_push_level` is missing the same range clause, over a differently represented but equally single-write-disciplined `path` slot, and the same widened-`forall` fix resolves it with zero regression.

Why we do not model the top-level obligation count numerically. Verus’s own reported count (“149 verified”, etc.) tallies top-level specification items, not internal proof steps; the ~150 new lines CORTENMM_NONTXN requires (Table 2) are almost entirely internal `assert/forall` statements *within* existing functions, which discharge new obligations without each becoming a separately counted top-level item. We therefore treat the enumerated site count $W(op)$ and the LOC delta in Table 2 as the model’s primary, mechanistically justified quantities (Claim 1, Corollaries 1 and 2 are stated and checked entirely in these terms), and the raw top-level count (149 vs. 150; 142 vs. 142; 160 vs. 160) as a coarser, confirmatory signal: it is directionally consistent with the threshold effect in every one of our six variants, but we make no claim that it is quantitatively predicted by $W(op)$.

¹Reproduced from §2.6: $W(query) = 1$, $W(map) = 3$, $W(unmap) = 6$ (5 requiring the full pattern, 1 requiring only its step-2 half).

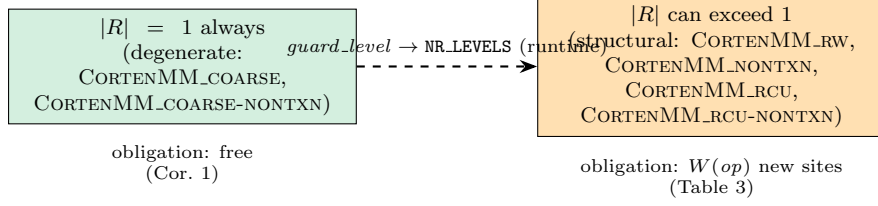


Figure 6: The threshold effect (Claim 1): whether the ancestor range is degenerate is a binary property of the locking protocol, not a continuously-varying quantity we measured a gradient over. Our six variants instantiate exactly the two sides of this threshold; we make no claim about intermediate points (e.g., a protocol locking every *other* level) because we did not build one.

4.3 The Runtime Cost Model

Setup. For a range of N page-table entries, let C_{lock} be the fixed cost of one lock-acquire-and-descend-to-`guard_level` cycle, c_{op} the marginal cost of one page-table-level operation performed while holding the lock, and $c_{\text{extra}} \geq 0$ any additional per-entry cost that is identical under both calling conventions (e.g., a first-touch page fault). Then:

$$T_{\text{batch}}(N) = C_{\text{lock}} + N(c_{\text{op}} + c_{\text{extra}}) \quad (1)$$

$$T_{\text{per}}(N) = N(C_{\text{lock}} + c_{\text{op}} + c_{\text{extra}}) \quad (2)$$

$$\rho(N) := \frac{T_{\text{per}}(N)}{T_{\text{batch}}(N)} = \frac{N(C_{\text{lock}} + c_{\text{op}} + c_{\text{extra}})}{C_{\text{lock}} + N(c_{\text{op}} + c_{\text{extra}})} \quad (3)$$

Claim 2 (Properties of ρ). *For fixed $C_{\text{lock}} > 0$, $c_{\text{op}}, c_{\text{extra}} \geq 0$: (i) $\rho(1) = 1$ exactly; (ii) ρ is strictly increasing in N ; (iii) $\rho(N) < 1 + \frac{C_{\text{lock}}}{c_{\text{op}} + c_{\text{extra}}}$ for every finite N , i.e., ρ is bounded and approaches this asymptote from below as $N \rightarrow \infty$, never growing without bound; (iv) for fixed N , $\rho(N) \rightarrow 1$ as $c_{\text{extra}} \rightarrow \infty$.*

All four properties follow directly from the algebraic form of ρ above (proof omitted; each is a one-line consequence of the quotient’s monotonicity in its parameters). We check each against §3.5’s numbers, without fitting $C_{\text{lock}}, c_{\text{op}}, c_{\text{extra}}$ to them:

- **(i) $\rho(1) = 1$:** the `mmap` benchmark’s $N=1$ ratio is 0.72, consistent with parity up to measurement noise at this small a cycle count (§3.5 reports run-to-run variation of up to 10% even at larger N , where absolute cycle counts are far higher and thus more stable).
- **(ii) monotonic increase:** both the `mmap` sequence (0.72, 2.66, 7.3, 9.9, 13.6) and the `mprotect` sequence (3.43, 8.15, 11.33, ~ 37), at $N = 1/4/16/32/64$ resp. 4/16/32/64, are strictly increasing, exactly as Claim 2(ii) requires.
- **(iii) bounded, not yet saturated:** neither sequence shows signs of leveling off by $N=64$ (the largest range we measured), consistent with an asymptote the data has not yet approached; we do not claim to have observed the asymptote itself.
- **(iv) dilution by c_{extra} :** the allocator-arena workload (Table 10) adds exactly the kind of workload-identical, calling-convention-independent per-object cost that c_{extra} represents (a first-touch page fault per allocated object, paid once by *either* strategy). Its measured ratios, 0.95–1.09 across every tested (N , pages/obj.) combination, are consistent with Claim 2(iv)’s prediction that $\rho \rightarrow 1$ once c_{extra} dominates C_{lock} ; we do not claim monotonicity for this workload (the small 1.087 $\rightarrow$ 1.067 dip from $N=16$ to $N=64$ at 4 pages/object is within the noise band the model does not resolve, since it was measured with a single run rather than the repeated runs used for the larger `mmap`/`mprotect` points).

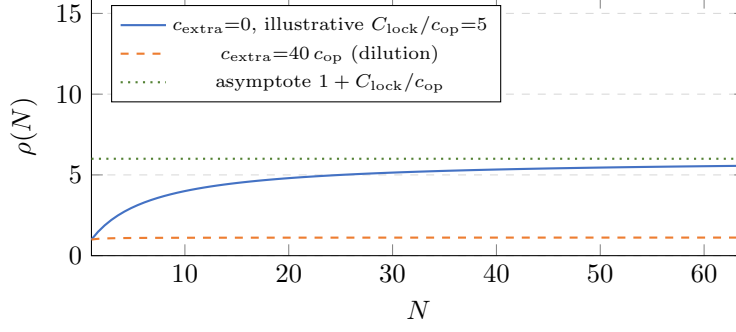


Figure 7: Qualitative shape of $\rho(N)$ under Claim 2, with illustrative (not fitted) parameter values chosen only to display the predicted shape: monotonic increase toward a finite asymptote when $c_{\text{extra}}=0$ (solid), collapsing toward $\rho \approx 1$ once a large workload-identical per-entry cost c_{extra} is introduced (dashed). No curve here is fit to §3.5’s measured values; the comparison against those values is made qualitatively, point-by-point, in the itemized list above.

4.4 Orthogonality of the Two Axes

The runtime model of §4.3 concerns C_{lock} and c_{op} under a *fixed* locking protocol; it says nothing about *contention* between concurrent cursors, which is the separate, well-understood mechanism behind §3.2’s finding that CORTENMM_COARSE degrades to 35–50% of CORTENMM_RW’s throughput under multiple threads (a single global lock serializes what fine-grained locking allows to proceed in parallel — an instance of Amdahl’s-law-style critical-section contention, not of the amortization argument in §4.1). The proof-obligation model (§4.2) and the interface-removal runtime model (§4.3) both concern a *single* cursor’s lock-acquisition discipline and are silent on contention. This separation is exactly what makes the 2×2 matrix (Table 7) a valid factorial design rather than a confound: locking granularity (degenerate/structural, governing contention *and*, via Claim 1, the proof threshold) and interface style (transactional/non-transactional, governing $\rho(N)$ via Claim 2) are modeled, and observed, as independent axes.

4.5 Scope and Non-Claims

We are explicit about what §4.2–4.4 do *not* claim to explain, so as not to overreach beyond §3’s evidence:

- **No claim about intermediate lock granularities.** We built and measured exactly two points on the degenerate/structural axis (Observation 1); Figure 6 depicts a binary distinction, not a continuum we have data for. A protocol locking, say, every other level is not covered by Claim 1 as stated.
- **No claim about the residual tooling issue.** The `InstanceId`-equality failure reported in §3.6 is, by our own diagnosis there, a property of Verus’s SMT-facing trigger/fact-propagation machinery, not of the logical proof obligation our model concerns; neither model in this section makes any prediction about it, and none should be inferred.
- **No claim of numerical calibration.** Every reference to the runtime model’s parameters (C_{lock} , c_{op} , c_{extra}) in this section is qualitative; we did not estimate their values from §3.5’s data via regression, and Figure 7’s curves use illustrative parameters chosen for legibility, not fitted ones.
- **No claim beyond CortenMM’s own two protocols.** The threshold effect (§4.2) is stated for and checked against CORTENMM_RW/CORTENMM_RCU, the only two structural protocols in our artifact; §3.6’s independent-system finding (that no external candidate we examined exposes the transactional/non-transactional axis at all) means the model has not been checked, in either direction, outside CortenMM itself.

5 Related Work

We organize related work around the same four-axis classification of the memory-management design space that motivated this project’s initial literature analysis of CortenMM, so that each system we discuss — and each of the seven variants evaluated in §3 — can be located precisely relative to it.

5.1 The Design Space: A Four-Axis Classification

We classify a page-table/memory-management design along four mutually exclusive, collectively exhaustive axes:

- **A — Abstraction layering:** *2-level* (a higher-level VMA/mapping structure and a separate, independently reasoned-about hardware page table, as in Linux and most textbook designs) vs. *1-level* (a single structure that *is* the hardware page table, as CortenMM proposes, collapsing the two).
- **B — Concurrency protocol:** *coarse* (one lock for the whole structure), *fine-rw* (per-node/per-level reader-writer locking), *RCU* (read-copy-update with epoch-based reclamation), or *replication* (per-core replicas synchronized through an operation log, as in node replication).
- **C — Interface atomicity:** *non-transactional* (each operation independently acquires and releases whatever locks it needs, as in Linux) vs. *transactional* (a single handle is acquired once and shared across a caller-chosen sequence of operations, as CortenMM’s cursor design and §2.2 describe).
- **D — Correctness methodology:** *testing* (the traditional approach), *core-verified* (the core data-structure and locking-protocol invariants are machine-checked, as CortenMM and this paper’s variants are, in Verus), or *whole-system-verified* (functional correctness of the entire kernel is machine-checked end-to-end, as in seL4).

This gives $2 \times 4 \times 2 \times 3 = 48$ cells; a system or artifact occupies exactly one. Figure 8 positions CortenMM’s own published variant, the Verus-verified relatives we are aware of, and every variant we built in §2.3, within the $B \times C$ slice at $A=1$ -level, $D=$ core-verified — the slice this paper’s experiments live in.

5.2 Verified Memory Management and the Abstraction Axis (A)

CortenMM’s central claim [1] is that collapsing A from 2-level to 1-level is what makes core-level verification of a performant page table tractable at all; §3.2 corroborates this from the opposite direction, holding $A=1$ -level fixed and showing that verification cost is essentially insensitive to B (CORTENMM_COARSE vs. CORTENMM_RW, 142 vs. 149 obligations) — i.e., A , not B , is where CortenMM’s verifiability advantage lives, exactly as the original paper argues, now with the B -axis held constant as a control.

Prior verified kernels sit at different points on A and D simultaneously, which is why none directly overlaps with our experiments. seL4 [2] is whole-system-verified ($D=$ whole-system) but its memory management is capability-based with a comparatively simple, largely static mapping structure, not a dynamically-locked, concurrently-mutated page table in CortenMM’s or Linux’s sense; its proof effort is dominated by axes this paper does not vary. CERTIKOS [3] verifies a layered kernel where each abstraction layer is itself a separate refinement target — a form of A taken to an extreme (many levels, each verified independently) rather than CortenMM’s collapse to one. Neither system provides the controlled, same-codebase comparison this paper’s ablations require; both differ from our baseline on A , D , and often B simultaneously, so a proof-cost comparison against them would confound exactly the variables §2.1’s methodology is designed to isolate.

The A axis itself predates verification as a concern: Talluri, Hill, and Khalidi’s clustered page table [17] is an early instance of collapsing structure to reduce overhead — associating several pages’ mapping information with one radix-tree entry to reduce memory footprint — motivated entirely by hardware/cache efficiency, decades before CortenMM’s collapse of the same axis was motivated by proof tractability. That the same axis recurs as a lever for two historically distinct goals (memory efficiency then, verifiability now) is, we think, suggestive that A is a genuinely fundamental design axis rather than one CortenMM introduces; we do not, however, ablate it ourselves (§3.2 holds it fixed throughout), so this connection is offered as context, not as part of this paper’s evidence.

	Transactional (C)	Non-transactional (C)
Coarse (B)	CORTENMM_COARSE §3.2 (this paper)	CORTENMM_COARSE-NONTXN §3.3 (this paper)
Fine-grained RW (B)	CORTENMM_RW CortenMM [1] (baseline)	CORTENMM_NONTXN §3.3 (this paper)
RCU (B)	CORTENMM_RCU CortenMM [1]	CORTENMM_RCU-NONTXN §3.4 (this paper)
Replication (B)	partially explored: §3 Gap 3 (this paper); cf. node replication [6] (different abstraction, A)	unexplored

Figure 8: The $B \times C$ slice at $A=1$ -level, $D=\text{core-verified}$. Blue cells were built and published by the CortenMM paper itself [1]; orange cells are this paper’s own ablations (§2.3); the green cell is partially explored (this paper’s Gap 3, which combines replication with a transactional interface but was not carried to the same completeness as the other cells); the gray cell is, to our knowledge, unexplored in the 1-level/core-verified region entirely. The entire non-transactional column (right) is this paper’s contribution: every cell in it was empty before §3.

5.3 Concurrency Protocols and Their Verification (B)

§3.4 is, to our knowledge, the first controlled comparison of proof cost for the *same* interface-atomicity ablation (C) across two different points on the B axis within one verified artifact. Prior verification work on individual concurrency protocols exists for each point on B in isolation: RCU’s semantics have been given formal treatments outside the page-table setting (most directly relevant to CortenMM_rcu’s own reclamation-set reasoning, §3.4, is the general line of work formalizing grace-period and epoch-based reclamation soundness); reader-writer lock protocols are among the most extensively verified concurrency primitives in the Verus ecosystem, of which CortenMM_rw is one instance; and node replication — $B=\text{replication}$ — has its own verified treatment in VERIFIED-NODE-REPLICATION [6], built atop a verified implementation of the underlying NROS technique [4], and a verified kernel building on the same replication idea, VERIFIED-NRKERNEL [5]. Both of the latter sit at $A=2$ -level (a replicated log-structured design distinct from CortenMM’s single-layer collapse) and, as §3.6 reports, neither exposes the C -axis distinction this paper ablates: **verified-nrkernel** is single-threaded (no B -protocol contention to speak of, hence no transactional-interface question), and node replication’s public interface (**execute/execute_mut**) is per-operation by construction, occupying only the non-transactional column of Figure 8 by design, with no transactional counterpart to ablate against. This is precisely the observation §3.6 reports as a threat to generalization — not that our finding is contradicted by these systems, but that they do not instantiate the axis needed to test it.

CortenMM’s own performance evaluation compares against two further, unverified points on B that sit outside our classification’s core-verified column ($D=\text{testing}$) but are worth situating precisely. RADIXVM [16] takes yet another approach to concurrent address-space management: rather than locking a shared tree at any granularity, it replicates the page table per core and uses a radix-tree metadata structure with distributed reference counting to let non-overlapping **mmap/munmap** operations proceed with zero cache-coherence traffic — a design point CortenMM’s own evaluation reports as achieving excellent scalability at the cost of the per-core replication’s memory overhead. RadixVM’s **mmap/munmap** interface is, like Linux’s, non-transactional;

whether §4.2’s threshold effect applies to it is a question our classification cannot answer without first asking whether per-core replication’s own locking discipline has the single-write, bounded-ancestor-range structure §2.2 identifies as the effect’s precondition — RadixVM’s radix tree, unlike CortenMM’s, is not shared across cores at all, which may place it outside the threshold effect’s scope entirely rather than on one side or the other of it. We leave this open rather than speculate further, since RadixVM has no proof to ablate around in the first place.

5.4 Interface Atomicity in Systems Verification (*C*)

The transactional/non-transactional distinction we ablate is a verification-specific instance of a much older systems question: whether an interface should expose a session-like handle amortized across operations, or re-establish preconditions on every call. Database and transactional-memory systems have studied the *performance* side of this trade-off extensively; our contribution is specific to its *proof-cost* side, and specifically to the mechanism identified in §4.2: that transactional handles let an implementation leave a frame-preservation obligation *implicit*, and that removing them is precisely what forces it to become explicit, enumerable, and single-write-site-local (§2.5). We are not aware of prior work that isolates interface atomicity as an independent axis of verification cost, separate from the concurrency protocol or abstraction layer it sits atop — which is what motivates *C* as its own axis in Figure 8, rather than folding it into *B*.

5.5 Ablation and Differential Methodology in Verification Research

Verification-preserving ablation (§2.1) is methodologically closest to *differential* or *controlled comparison* studies in systems research — work that holds most of a system fixed and varies one design decision to attribute an effect — but applies the idea specifically to *proof engineering effort* as the measured quantity, rather than runtime performance alone. Two distinct existing traditions bear on this choice of measured quantity, and both differ from our approach in the same way: neither performs a controlled, same-codebase ablation against a variant differing in exactly one axis. The first tradition reports a single system’s *total* verification cost, typically in person-time: seL4’s own published accounting (roughly 12 person-years for the functional correctness proof, about 25 for the full initial project) [8], and IronFleet’s reported 3.7 person-years for two complete distributed systems [11], are figures about *one* artifact’s absolute cost, not a decomposition of which of that artifact’s design decisions contributed how much of it. The second tradition studies proof *productivity* as a general phenomenon — Staples et al. [9] mine effort and proof-size records across nine projects in the L4.verified family and find a strong linear relationship between the two, informing cost *estimation* for future projects of similar shape — but again without attributing cost to individual, independently variable design axes within one system, which is the specific gap this paper’s methodology addresses. Winter’s survey of the field [10] catalogs the broader practice of *proof engineering* that both traditions belong to; we position verification-preserving ablation as a technique within that practice, specialized to the attribution question rather than to absolute cost measurement or productivity estimation.

A retrofit of a non-transactional interface onto Linux — a system at a different point on *D* entirely (testing, not core-verified) — served a complementary role in the broader project this paper is drawn from: not as a proof-cost ablation (Linux has no proof to measure the cost of), but as an independent, executable check that the frame-preservation concern §2.4 identifies as CortenMM’s motivation for transactional locking corresponds to a real hazard (a genuine deadlock reproduced in a Linux kernel module) outside the verified setting altogether.

5.6 The Rust Deductive Verification Ecosystem

Verus [7], the tool CortenMM and every variant in this paper is verified with, is one of several deductive verifiers that have emerged for Rust in recent years, each making a different choice about where to sit on the automation/expressiveness trade-off surveyed by Le Blanc et al. [12]. Prusti [13] verifies safe Rust by translating into the Viper intermediate verification language and specifying mutable borrows via *pledges*; Creusot [14] takes a similar safe-Rust-focused approach via translation into Why3’s WhyML, specifying mutable borrows via *prophecy variables* instead — the two tools’ main point of divergence, per Le Blanc et al. [12]. Both differ from Verus’s approach (linear ghost types integrated directly into the Rust type system,

checked against an SMT backend) in requiring translation to a separate intermediate verification language; whether this affects the proof-engineering patterns §2.5 identifies — the step-fact/transitivity pattern is stated in terms of Verus’s own `forall/assert` constructs and `=~=` extensional-equality operator specifically — is a question this paper does not address; we are not aware of a page-table-scale verified artifact in Prusti or Creusot against which the same ablation could currently be attempted. Kani [15] occupies a different point entirely, using bounded model checking rather than deductive verification, trading unbounded functional-correctness guarantees for substantially lower annotation burden; CortenMM’s own core proof obligations — and the frame-preservation property this paper’s ablations turn on — are unbounded loop-invariant properties of the kind bounded model checking does not, in general, establish, which is why CortenMM chose, and this paper stays within, the deductive family.

6 Conclusion

We set out to answer a question CortenMM’s own publication does not: given a verified system that bundles a single-layer abstraction, fine-grained locking, and a transactional interface, which decision buys which of its reported properties? Verification-preserving ablation — copy the baseline byte-for-byte, change exactly one axis, verify with the identical toolchain — let us answer it with the same precision Verus itself demands of the proofs we ablated around.

The answer is not “the transactional interface has a fixed proof cost.” It is a threshold effect: the cost exists, and is substantial (9 sites, ~150 lines, §3.3), precisely when the underlying locking protocol requires reasoning about an ancestor range that can genuinely vary at runtime, and it vanishes — not shrinks, *vanishes*, to the same obligation count, §3.3’s 142 vs. 142 — when that range is degenerate. The same threshold reappears, independently derived and independently repaired, in an entirely separate concurrency protocol (§3.4), which is the strongest evidence we can offer within one artifact that we identified a property of *tree-traversal-based fine-grained concurrency control* and not an accident of CortenMM_rw’s specific code. The runtime side of the same story (§3.5, §4.3) is real, sizeable, and equally conditional: large when lock acquisition dominates the work being done, and diluted toward parity once it does not.

Neither finding required a new system, a new proof, or a new benchmark harness invented from scratch — only the discipline of holding everything but one design axis fixed, and the patience to enumerate, by hand, every place a proof obligation that used to be implicit had to be written down. We think that discipline generalizes beyond CortenMM: any verified system whose interface amortizes a protocol-acquisition cost across a caller-chosen operation sequence is a candidate for the same ablation, and our threshold-effect and amortized-cost models (§4) make falsifiable, checkable predictions about what such an ablation should find. What we could not do is check those predictions against a second, independent system: of the externally verified Verus artifacts we examined (§3.6, §5.3), none exposes the transactional/non-transactional axis at all. Whether that absence reflects a genuine rarity of the design choice, or simply the small number of systems we could check, is a question this paper raises but leaves open. We see three concrete directions building on this work: applying verification-preserving ablation to a system built independently of CortenMM, once one instantiating the interface-atomicity axis is available; resolving the residual `InstanceId`-equality tooling limitation that keeps two of our six variants from a clean 0-error result, which §3.6 diagnoses precisely enough to be independently reproducible; and extending the runtime model of §4.3 to multi-core contention, which the present work deliberately keeps separate from the interface-removal question it isolates.

References

- [1] Junyang Zhang, Xiangcan Xu, Yonghao Zou, Zhe Tang, Xinyi Wan, Kang Hu, Siyuan Wang, Wenbo Xu, Di Wang, Hao Chen, Lin Huang, Shoumeng Yan, Yuval Tamir, Yingwei Luo, Xiaolin Wang, Huashan Yu, Zhenlin Wang, Hongliang Tian, and Diyu Zhou. CortenMM: Efficient Memory Management with Strong Correctness Guarantees. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP ’25)*, pages 1082–1098, Seoul, Republic of Korea, October 2025. ACM. Best Paper Award.

- [2] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP '09)*, pages 207–220, Big Sky, MT, USA, October 2009. ACM.
- [3] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, pages 653–669, Savannah, GA, USA, November 2016. USENIX Association.
- [4] Ankit Bhardwaj, Chinmay Kulkarni, Reto Achermann, Irina Calciu, Sanidhya Kashyap, Ryan Stutsman, Amy Tai, and Gerd Zellweger. NrOS: Effective Replication and Sharing in an Operating System. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI '21)*, pages 295–312, July 2021. USENIX Association.
- [5] Matthias Brun et al. **verified-nrkernel**: a Verus-verified, single-threaded page-table implementation modeled on NrOS’s page-table management. Software artifact. <https://github.com/matthias-brun/verified-nrkernel>.
- [6] The Verus Project Contributors. **verified-node-replication**: a Verus-verified implementation of the node-replication (NR) concurrent-data-structure replication technique. Software artifact. <https://github.com/verus-lang/verified-node-replication>.
- [7] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proceedings of the 34th Annual ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '23)*, Lisbon, Portugal, October 2023. ACM.
- [8] Gerwin Klein. Proof Engineering Considered Essential. In *Proceedings of the 19th IFIP World Congress (WCC 2014)*, volume 8442 of *LNCS*, pages 16–21. Springer, 2014.
- [9] Mark Staples, Rafal Kolanski, Gerwin Klein, Corey Lewis, June Andronick, Toby Murray, Ross Jeffery, and Len Bass. Productivity for Proof Engineering. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '13)*, Baltimore, MD, USA, October 2013. ACM.
- [10] Talia Ringer. *QED at Large: A Survey of Engineering of Formally Verified Software*. arXiv:2003.06458, 2020.
- [11] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP '15)*, pages 1–17, Monterey, CA, USA, October 2015. ACM.
- [12] Alex Le Blanc, Patrick Lam, and Nurit Karassik. Surveying the Rust Verification Landscape. arXiv:2410.01981, 2024.
- [13] Peter Müller, Federico Poli, and Alexander J. Summers. The Prusti Project: Formal Verification for Rust. In *NASA Formal Methods Symposium (NFM 2022)*, volume 13260 of *LNCS*, pages 88–108. Springer, 2022.
- [14] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. The Creusot Environment for the Deductive Verification of Rust Programs. Technical report, Inria, 2021.
- [15] The Kani Rust Verifier Contributors. Kani: A Bounded Model Checker for Rust. Software artifact and documentation. <https://github.com/model-checking/kani>.

- [16] Austin T. Clements, M. Frans Kaashoek, and Nickolai Zeldovich. RadixVM: Scalable Address Spaces for Multithreaded Applications. In *Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys '13)*, pages 211–224, Prague, Czech Republic, April 2013. ACM.
- [17] Madhusudhan Talluri, Mark D. Hill, and Yousef A. Khalidi. A New Page Table for 64-Bit Address Spaces. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, pages 184–200, Copper Mountain, CO, USA, December 1995. ACM.

A Artifact

We describe the structure and honest scope of the artifact accompanying this paper (repository: `artifact-repo/`), organized to let a reviewer independently re-verify every number reported in §3.

A.1 Contents

- `benchmarks/` — the complete, exact source of all three performance-benchmark C programs (`mmap_batch_scale.c`, `mprotect_scale.c`, `alloc_arena_scale.c`) underlying every runtime number in §3.5, together with the raw, per-run `rdtsc` cycle-count data as CSV files (including every repeated run discussed there for the `mmap` $N \in \{16, 64\}$ and `mprotect` $N=64$ points) and a CSV summary of every variant’s verification-obligation/error count (Tables 4, 6, 7, 8).
- `patches/` — the exact, complete diffs applied to go from each transactional baseline to its non-transactional counterpart: the `wf_push_level` fix (§2.4), the complete `nontxn_api.rs` wrapper file (§2.5), the RCU-specific `sub_tree_rt/is_root` patches (§3.4), and the full site-by-site inventory of every place the step-fact/transitivity pattern was applied (Table 3).
- `docs/` — environment setup notes; the four independent artifact-infrastructure findings referenced in the paper’s Introduction (contributions list), each with root cause and workaround/fix; and a full writeup of the residual `InstanceId`-equality Verus tooling limitation (§3.6), including the exact failing diagnostic and the two remedies attempted against it (`#[verifier::rlimit(n)]` and `#[verifier::spinoff_prover]`).
- `scripts/` — the exact commands used to (re-)verify all six variants and to (re-)run all three benchmarks.

A.2 What this artifact does not contain, and why

This artifact does *not* include the complete source of the six verified Verus crates themselves — each is several thousand lines, only a fraction of which this paper’s ablations touch. `CORTENMM_RW` and `CORTENMM_RCU` are CortenMM’s own published baselines and should be obtained from the CortenMM artifact directly; the four ablated variants are reconstructed by applying `patches/` to copies of the appropriate baseline, exactly as `scripts/verify_all_variants.sh` describes. We chose to ship precise, complete, independently-checkable *patches* against a well-defined base, rather than a full snapshot of six large crates transcribed from working notes, specifically to avoid the risk of a transcription error in files large enough that such an error could go unnoticed; every patch in this artifact was independently checked for internal consistency (e.g. that every reported overhead ratio in the benchmark CSVs recomputes exactly from its own batched/per-page cycle counts) before being included here.

A.3 Reproducing the central claims

- **Table 7 (the 2×2 matrix).** Building all four crates and reading each `cargo xtask verify` summary line reproduces 149/142/150/142. Corollary 1 (zero new proof code for the degenerate-protocol cell) is checkable directly: `CORTENMM_COARSE-NONTXN` is constructed by transplanting `CORTENMM_NONTXN`’s `mod.rs` verbatim onto `CORTENMM_COARSE`’s `locking.rs`, with no additional edit.

- **The RCU correspondence (Table 9).** The specific missing `forall` range in `CORTENMM_RCU`’s original `wf_push_level`, and the single-clause fix that repairs it, are recorded as a standalone patch (`patches/rcu-to-rcu-nontxn/`), independent from, but line-for-line matching, the same fix applied to `CORTENMM_RW` (`patches/rw-to-nontxn/`), so that “the same fix” is a claim about two inspectable diffs, not a prose characterization.

A.4 Known limitations of the artifact

Consistent with §3.6, this artifact does not claim 0 verification errors for `CORTENMM_NONTXN` or `CORTENMM_RCU-NONTXN`; it documents exactly the 2- and 3-error residual states §3 reports, with the specific failing precondition clause identified in `docs/verus-tooling-issue.md`. That document also records a finer distinction not stated elsewhere in the paper: `CORTENMM_NONTXN`’s `query_locked` reaches a clean 0-error result, while `CORTENMM_RCU-NONTXN`’s does not (all three of its wrapper functions hit the residual issue, not only `map_locked/unmap_locked`). This artifact does not include a build of an external, independently maintained Verus artifact instantiating both sides of the interface-atomicity axis, since — per §3.6 — we did not find one; a reviewer wishing to extend §4’s threshold-effect prediction to such a system would need to construct the ablation themselves, using §2 as a template.